qu.dev

Project Documentation

Spring Boot Web Tool

Author	Asfar Muhammed N S
Repository	github.com/Asfar-07/qu.dev
Version	0.0.1-SNAPSHOT
Language	Java 21
Framework	Spring Boot 4.0.5
Date	April 2026

1. Project Overview

qu.dev is a full-stack web application built with Spring Boot. It serves as a server-side rendered web platform using Thymeleaf templates with a MySQL database backend. The application follows the standard MVC (Model-View-Controller) architecture provided by the Spring framework.

The application is designed to be a developer-oriented platform (implied by the name "qu.dev"), leveraging modern Java tooling with persistence via JPA/Hibernate.

2. Technology Stack

2.1 Core Technologies

Dependency	Version / Scope	Purpose
`Java`	21 (LTS)	Primary programming language
`Spring Boot`	4.0.5	Application framework and auto-configuration
`Spring MVC`	Included	Model-View-Controller web layer
`Spring Data JPA`	Included	ORM / database persistence layer
`Thymeleaf`	Included	Server-side HTML templating engine
`MySQL`	Runtime	Primary production relational database
`H2 Database`	Runtime	In-memory database for development/testing
`Lombok`	Optional	Boilerplate reduction (getters, setters, builders)
`Spring DevTools`	Included	Hot reload during development
`Maven`	Wrapper included	Build tool and dependency management

2.2 Frontend Technologies

The project's language breakdown indicates a significant frontend component built with web technologies:

Dependency	Version / Scope	Purpose
`HTML`	41.3% of codebase	Page structure and Thymeleaf templates
`CSS`	30.8% of codebase	Styling and responsive layout
`Java`	24.5% of codebase	Backend business logic and controllers
`JavaScript`	3.4% of codebase	Client-side interactivity

3. Architecture

3.1 Application Architecture

qu.dev follows the standard Spring Boot MVC architecture:

- Model — JPA entities mapped to the MySQL database using Spring Data JPA and Hibernate ORM
- View — Thymeleaf HTML templates rendered server-side with dynamic data
- Controller — Spring MVC @Controller classes handling HTTP routes and binding model data to views

The application uses a layered architecture pattern:

- Presentation Layer: Thymeleaf templates (HTML/CSS/JS)
- Web Layer: Spring MVC Controllers
- Service Layer: Business logic (Spring @Service beans)
- Repository Layer: Spring Data JPA repositories
- Data Layer: MySQL (production) / H2 (development)

3.2 Project Structure

The project follows the standard Maven directory layout for a Spring Boot application:

```
qu.dev/
├── .mvn/wrapper/          # Maven wrapper configuration
├── src/
│   ├── main/
│   │   ├── java/com/example/qudev/
│   │   │   ├── QudevApplication.java   # Entry point
│   │   │   ├── controller/             # MVC Controllers
│   │   │   ├── model/                  # JPA Entities
│   │   │   ├── repository/             # Spring Data Repositories
│   │   │   └── service/                # Business Logic
│   │   └── resources/
│   │       ├── application.properties  # App configuration
│   │       ├── static/                 # CSS, JS, images
│   │       └── templates/              # Thymeleaf HTML templates
│   └── test/                           # Unit & integration tests
├── pom.xml                             # Maven build file
└── mvnw / mvnw.cmd                     # Maven wrapper scripts
```

4. Dependencies

4.1 Core Dependencies

The following dependencies are defined in pom.xml:

Dependency	Version / Scope	Purpose
`spring-boot-starter-web`	Compile	Embeds Tomcat, enables REST & MVC controllers
`spring-boot-starter-data-jpa`	Compile	Hibernate ORM + Spring Data repositories
`spring-boot-starter-thymeleaf`	Compile	Server-side HTML template rendering
`spring-boot-starter-webmvc`	Compile	Spring Web MVC framework
`spring-boot-devtools`	Compile	Auto-restart on code changes during dev
`spring-boot-h2console`	Compile	H2 web console for in-memory DB inspection

4.2 Database Dependencies

Dependency	Version / Scope	Purpose
`mysql-connector-j`	Runtime	JDBC driver for MySQL production database
`h2`	Runtime	Embedded in-memory database for local development

4.3 Utility Dependencies

Dependency	Version / Scope	Purpose
`lombok`	Optional (compile-time)	Generates getters, setters, constructors, builders via annotations

4.4 Test Dependencies

Dependency	Version / Scope	Purpose
`spring-boot-starter-data-jpa-test`	Test	Testing support for JPA repositories
`spring-boot-starter-thymeleaf-test`	Test	Testing utilities for Thymeleaf views

`spring-boot-starter-webmvc-test`	Test	MockMvc and controller test support

5. Getting Started

5.1 Prerequisites

Ensure the following are installed on your development machine:

- Java Development Kit (JDK) 21 or later
- Maven 3.8+ (or use the included mvnw wrapper)
- MySQL 8.0+ server running locally or remotely
- Git
- An IDE such as IntelliJ IDEA or Eclipse (recommended)

5.2 Clone the Repository

```
git clone https://github.com/Asfar-07/qu.dev.git
cd qu.dev
```

5.3 Database Configuration

Configure your MySQL connection in src/main/resources/application.properties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/qudev
spring.datasource.username=<your-username>
spring.datasource.password=<your-password>
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

For local development using H2 (no MySQL required), use:

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.h2.console.enabled=true
```

5.4 Build and Run

Using the Maven wrapper (no Maven installation required):

```
# On Linux / macOS
./mvnw spring-boot:run

# On Windows
mvnw.cmd spring-boot:run
```

Or build a JAR and run it:

```
./mvnw clean package
java -jar target/qudev-0.0.1-SNAPSHOT.jar
```

The application starts on http://localhost:8080 by default.

6. Configuration Reference

Key configuration properties for the application:

Property	Default	Description
`server.port`	8080	HTTP port the app listens on
`spring.jpa.hibernate.ddl-auto`	update	Schema strategy: none / update / create
`spring.jpa.show-sql`	false	Logs SQL queries to console
`spring.h2.console.enabled`	false	Enable H2 browser console at /h2-console
`spring.thymeleaf.cache`	true	Disable in dev for hot-reload of templates
`spring.devtools.restart.enabled`	true	Auto-restart on classpath changes

7. Key Features & Capabilities

7.1 Web Layer

- Server-Side Rendering (SSR) via Thymeleaf — no client-side framework required
- Spring MVC controllers handle HTTP GET/POST routing and form submissions
- Static assets (CSS, JS, images) served from src/main/resources/static/
- DevTools enable automatic restart and LiveReload during development

7.2 Data Layer

- Spring Data JPA repositories with CRUD operations out of the box
- Hibernate ORM maps Java entity classes to database tables automatically
- Dual database support: MySQL for production, H2 for rapid local development
- DDL auto-management: schema is updated automatically on application start

7.3 Developer Experience

- Lombok eliminates boilerplate — use @Data, @Builder, @NoArgsConstructor on entity classes
- Spring DevTools provides hot-reload without restarting the server manually
- H2 Console at /h2-console for in-browser SQL queries during development
- Maven wrapper included — no local Maven installation required to build

8. Testing

The project includes test dependencies for three main testing concerns:

Repository / Data Layer Tests

Using spring-boot-starter-data-jpa-test, you can write @DataJpaTest tests that spin up an in-memory H2 database and test repository queries in isolation:

```
@DataJpaTest
class UserRepositoryTest {
    @Autowired UserRepository repo;
    @Test void testFindByEmail() { /* ... */ }
}
```

Controller / Web Layer Tests

Using spring-boot-starter-webmvc-test provides MockMvc for testing controllers without a running server:

```
@WebMvcTest(UserController.class)
class UserControllerTest {
    @Autowired MockMvc mockMvc;
    @Test void testHomePage() throws Exception {
        mockMvc.perform(get("/")).andExpect(status().isOk());
    }
}
```

Thymeleaf View Tests

spring-boot-starter-thymeleaf-test allows validating that templates render correctly with given model data.

Run all tests with:

```
./mvnw test
```

9. Build & Deployment

9.1 Build Commands

Dependency	Version / Scope	Purpose
`./mvnw clean`	-	Remove previous build artifacts
`./mvnw compile`	-	Compile Java sources
`./mvnw test`	-	Run all unit and integration tests
`./mvnw package`	-	Produce executable JAR in target/
`./mvnw spring-boot:run`	-	Run the app directly via Maven
`./mvnw clean install`	-	Full build, test, and install to local repo

9.2 Production Deployment

After packaging, the self-contained JAR includes an embedded Tomcat server and can be deployed anywhere Java 21 is available:

```
# Build
./mvnw clean package -DskipTests

# Deploy (Linux server)
java -jar target/qudev-0.0.1-SNAPSHOT.jar \
     --spring.datasource.url=jdbc:mysql://prod-db:3306/qudev \
     --spring.datasource.username=prod_user \
     --spring.datasource.password=secret
```

For containerized deployments, a minimal Dockerfile would look like:

```
FROM eclipse-temurin:21-jre
COPY target/qudev-0.0.1-SNAPSHOT.jar app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

10. Lombok Usage Guide

Lombok is configured in this project to reduce boilerplate. Common annotations used on entity and DTO classes:

Dependency	Version / Scope	Purpose
`@Data`	-	Generates getters, setters, equals, hashCode, toString
`@Builder`	-	Generates a builder pattern for the class

`@NoArgsConstructor`	-	Generates a no-argument constructor
`@AllArgsConstructor`	-	Generates a constructor with all fields
`@Getter / @Setter`	-	Individual getter or setter generation
`@Slf4j`	-	Injects a logger field: private final Logger log = ...

Note: Lombok is excluded from the final JAR (it is compile-time only). The maven-compiler-plugin is configured with annotationProcessorPaths to ensure Lombok runs during compilation.

11. Contributing

To contribute to the qu.dev project:

- Fork the repository on GitHub
- Create a feature branch: git checkout -b feature/my-feature
- Make your changes and write tests where appropriate
- Ensure all tests pass: ./mvnw test
- Commit your changes: git commit -m 'Add: description of change'
- Push to your fork: git push origin feature/my-feature
- Open a Pull Request against the main branch

12. Glossary

Dependency	Version / Scope	Purpose
`Spring Boot`	-	Opinionated framework for building stand-alone Spring applications
`Thymeleaf`	-	Java-based XML/HTML template engine for server-side rendering
`JPA`	-	Java Persistence API — standard ORM specification in Java
`Hibernate`	-	JPA implementation that translates Java objects to SQL
`H2`	-	Lightweight in-memory Java database, ideal for testing and local dev
`Lombok`	-	Annotation processor that generates repetitive Java boilerplate
`MVC`	-	Model-View-Controller — architectural pattern used by Spring Web
`DDL`	-	Data Definition Language — SQL statements for schema management

`DXA`	-	Twips unit used in OOXML (1440 DXA = 1 inch)
`SNAPSHOT`	-	Maven version suffix indicating an in-development build